ExecuteSpec Web UI Persona Audit

Screenshot-backed product assessment of the web Console and agent experience, including a strict comparison against OpenAI Codex Web, Claude Code, Cursor Agent/Bugbot, Devin, and Replit Agent.

Verdict	**The web UI is a credible operator console, but it is not yet a complete replacement surface for Codex/Claude-style coding work. The missing product proof is not backend capability; it is visible workflow clarity: prompt -> permissions -> live work -> diff -> verify -> evidence -> revert.**

Audit Basis

Environment	Local Vite frontend at 127.0.0.1:5173 backed by deployed dev API at dev.executespec.ai. This is real app/API behavior, not a static mockup.
Screenshot set	12 accepted screenshots: login, agent, settings, chats drawer, compose, work list, run detail, projects, setup, command palette, mobile agent, mobile run detail.
Code cross-check	Console shell and agent implementation show route-based nav, command palette, labeled top-bar controls, agent sessions, gates, capabilities, managed-AI state, and pinned run polling.
Limits	No full keyboard traversal, screen-reader pass, measured contrast audit, or production deploy smoke was performed in this report. Accessibility findings are visible risks, not full WCAG claims.

1. Persona Findings

Solo dev replacing Claude Code/Codex	2.5 / 5	Agent entry, Start work, run detail, and command palette are present.	The core local coding loop is not explicit. Permissions, diff, verify, evidence, and revert are not presented as one guided flow.
Senior engineer on an existing repo	3 / 5	Run evidence tabs, local execution label, provider/cost details, and project surfaces are useful.	Code inspection and source-level proof are less immediate than terminal/IDE-native competitors.
Engineering manager / org evaluator	4 / 5	Setup, team/provider/billing grouping, run history, and cost posture are stronger than most coding-agent competitors.	The UI still makes the product look like an internal console unless proof and outcomes become more visible.
Security/platform owner	3 / 5	Local execution and provider/cost metadata are visible; top-level controls are mostly labeled.	The web UI does not yet make command approval, file permission, redaction, and revert boundaries obvious enough.

- **Best current use case:** governance-heavy run tracking and operational review of work already submitted.
- **Weakest current use case:** hands-on replacement for Claude Code/Codex where the user expects tight local-loop confidence.
- **Most valuable visible asset:** run detail. It has the right raw material: status, execution mode, provider, cost, tabs, and artifacts.

2. Competitor Comparison

Scores are product-surface scores for the audited job: can a developer or manager use the visible UI to trust and control coding-agent work. They are not model-quality scores.

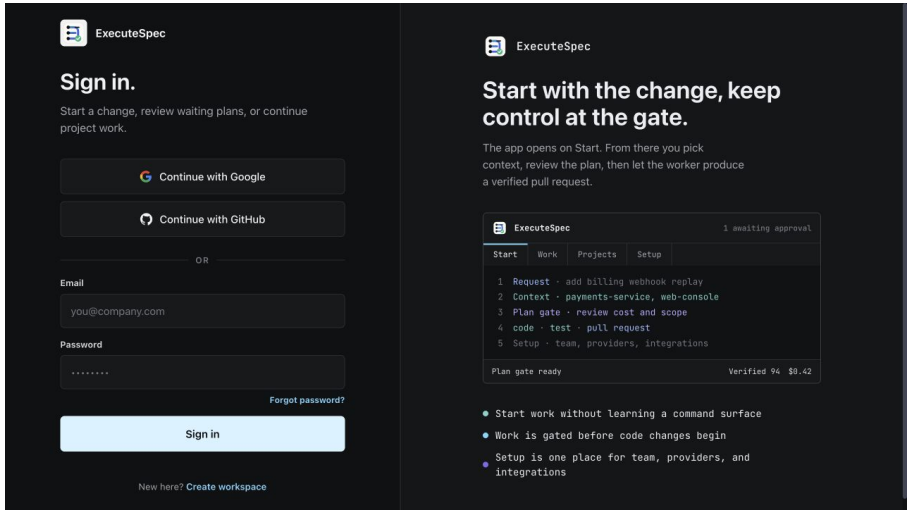
OpenAI Codex Web	4 / 5	Clearer cloud task delegation and PR/code-review framing. Stronger first-use task abstraction.	ExecuteSpec exposes more cost, plan, provider, org, and run evidence surfaces.
Claude Code	4.5 / 5	Terminal-native permission and tool transparency. Better fit for immediate local coding replacement.	ExecuteSpec has a richer web governance/run-management shell.
Cursor Agent / Bugbot	4 / 5	IDE context, background agent entry, and code-review adjacency are closer to where developers work.	ExecuteSpec is stronger as a central control plane across runs, projects, plans, cost, and setup.
Devin	4 / 5	Autonomous-engineer narrative and browsable workspace are easier to understand quickly.	ExecuteSpec can win on policy, local execution, cost visibility, and multi-surface governance if exposed better.
Replit Agent	3.5 / 5	Web-native build/deploy loop gives faster artifact gratification.	ExecuteSpec has stronger enterprise repo/governance intent and a better fit for serious codebase workflows.
ExecuteSpec Web today	3 / 5	N/A	Good bones: dense console, real run evidence, local execution label, cost/provider details, command palette, setup/governance.

Strict read: ExecuteSpec should not try to out-IDE Cursor or out-terminal Claude Code from the web. Its web advantage should be the control plane: proof, cost, policies, replay, gates, and team-visible outcomes. Today those pieces exist, but the replacement loop is not stitched together tightly enough.

3. Scored Parameters

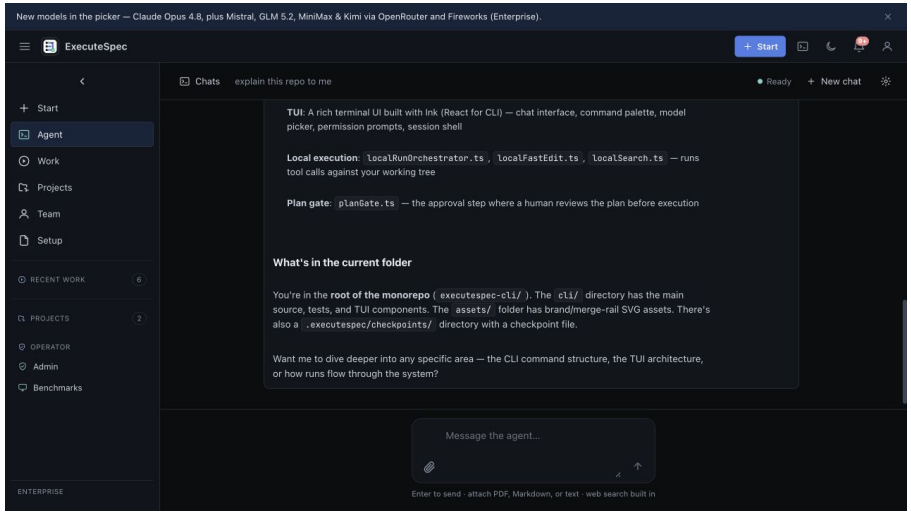
Task entry clarity	3 / 5	Login and Compose explain starting work, but agent entry opens an old chat and does not frame local-hands coding.	P1
Local-hands clarity	2 / 5	Run detail labels local execution, but agent/compose do not show permission or local machine boundaries upfront.	P0
Live work cockpit	2.5 / 5	Run detail has tabs; agent can pin active runs, but screenshots do not show a rich live cockpit.	P0
Diff / verify / evidence / revert	2 / 5	Evidence and tests tabs exist; primary action row does not make diff/verify/revert the default path.	P0
Governance and setup	4 / 5	Setup groups team, providers, billing, projects, work, and notifications well.	P2
Cost and model transparency	4 / 5	Run detail exposes provider, worker, cost, credits, and managed-AI hooks exist in agent code.	P2
Mobile usability	2 / 5	Mobile works but banner and dense content push proof/actions below the fold.	P1
Keyboard/operator ergonomics	3.5 / 5	Command palette and nav are strong; local-hands commands are missing.	P2

4. Evidence Walkthrough



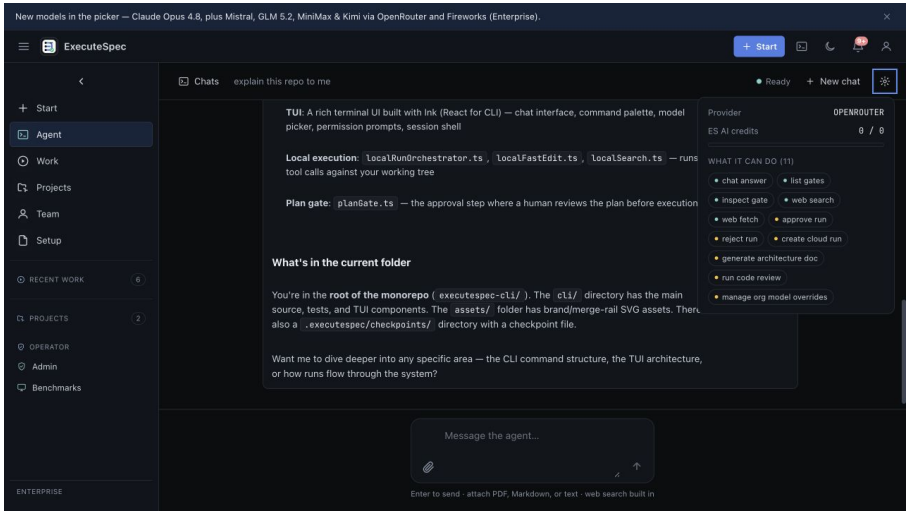
1. Login and positioning

- Good: clear product thesis and simple auth choices.
- Risk: first-screen proof is marketing-heavy; no visible local-hands workflow.



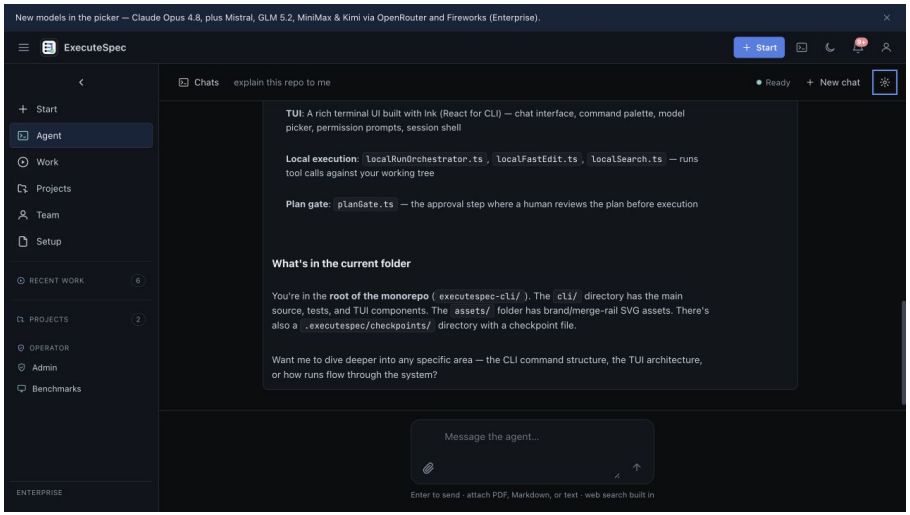
2. Agent entry with existing chat

- Good: real chat-first agent shell with composer and persistent nav.
- Risk: old transcript as default hurts first-run confidence.



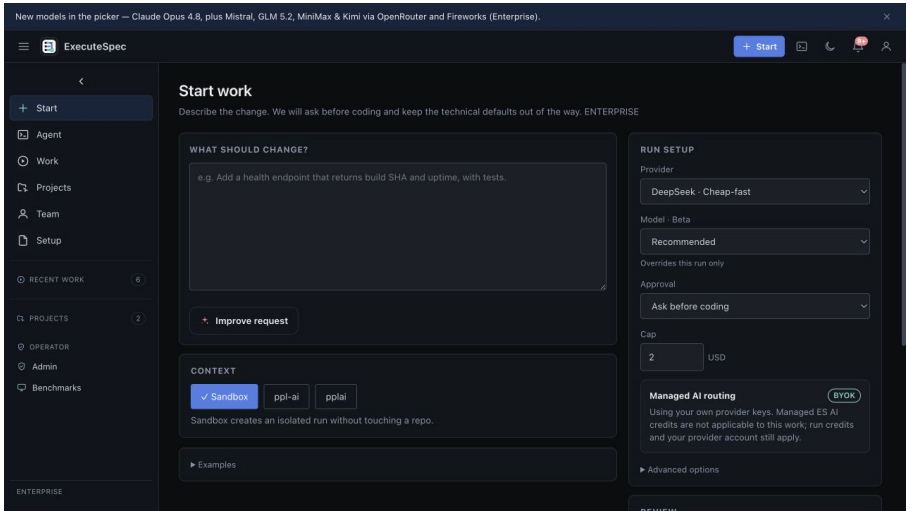
3. Agent settings and usage

- Good: exposes internal execution/usage concepts.
- Risk: too internal for trust; no permission boundary explanation.



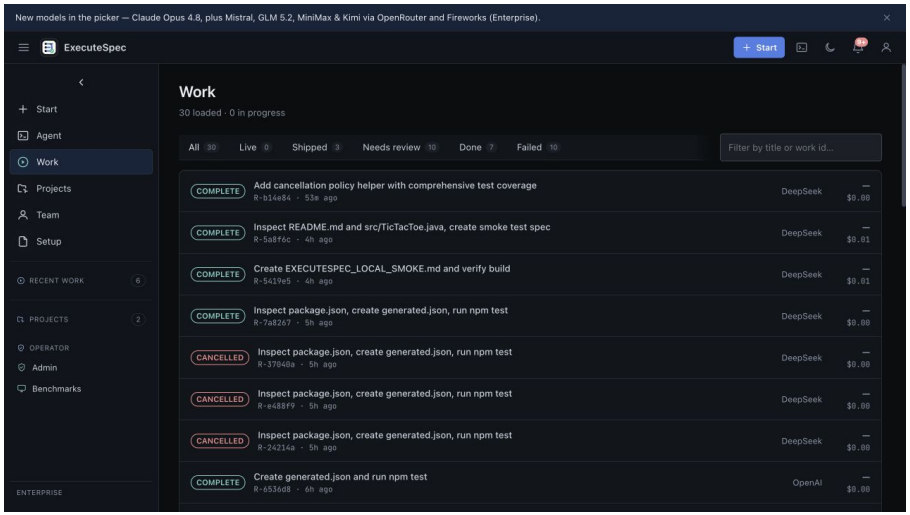
4. Chats drawer

- Good: continuity and session access.
- Risk: weak triage/status; hard to know which chat matters.



5. Start work / compose

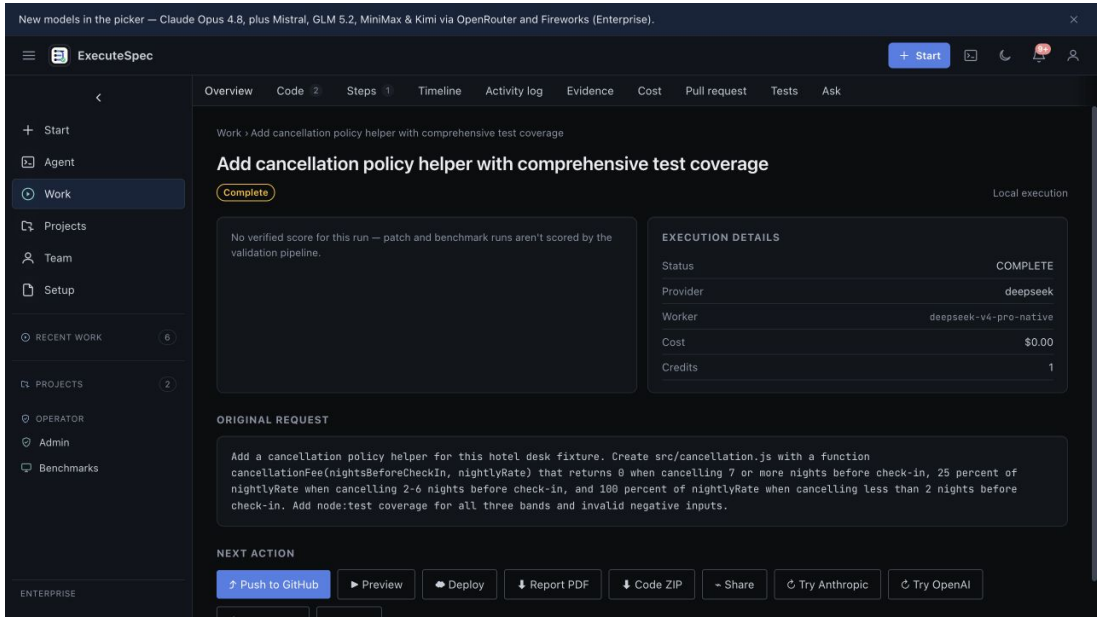
- Good: start form is functional and compact.
- Risk: target loop is hidden; local execution is just a setting, not a guided path.



6. Work list

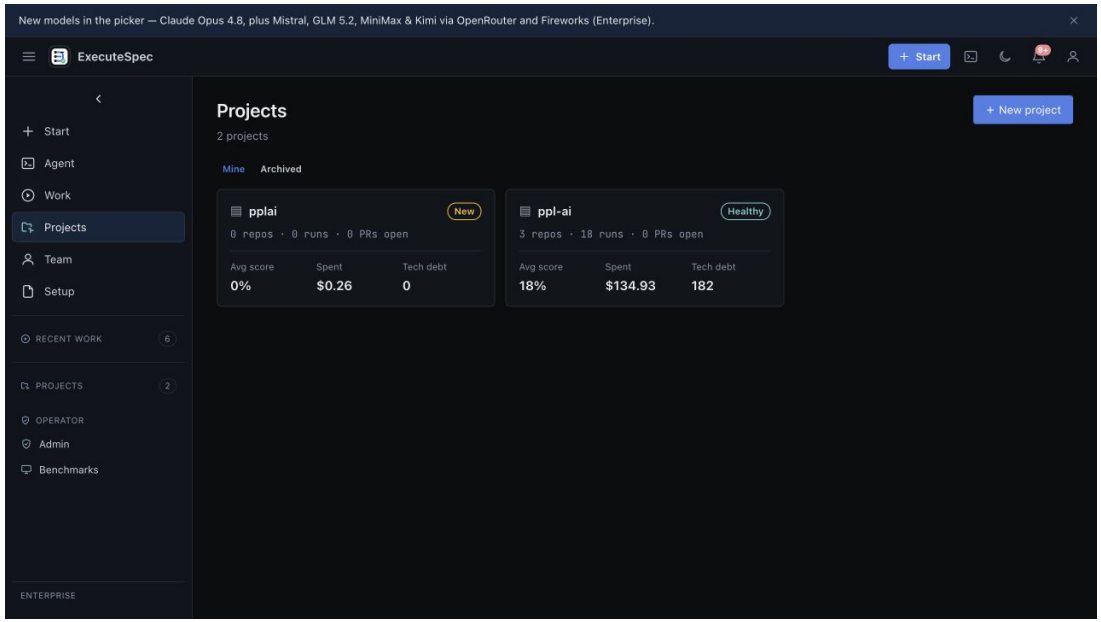
- Good: dense operational list and status scanning.
- Risk: small text and proof state are hard to parse quickly.

5. Evidence Walkthrough Continued



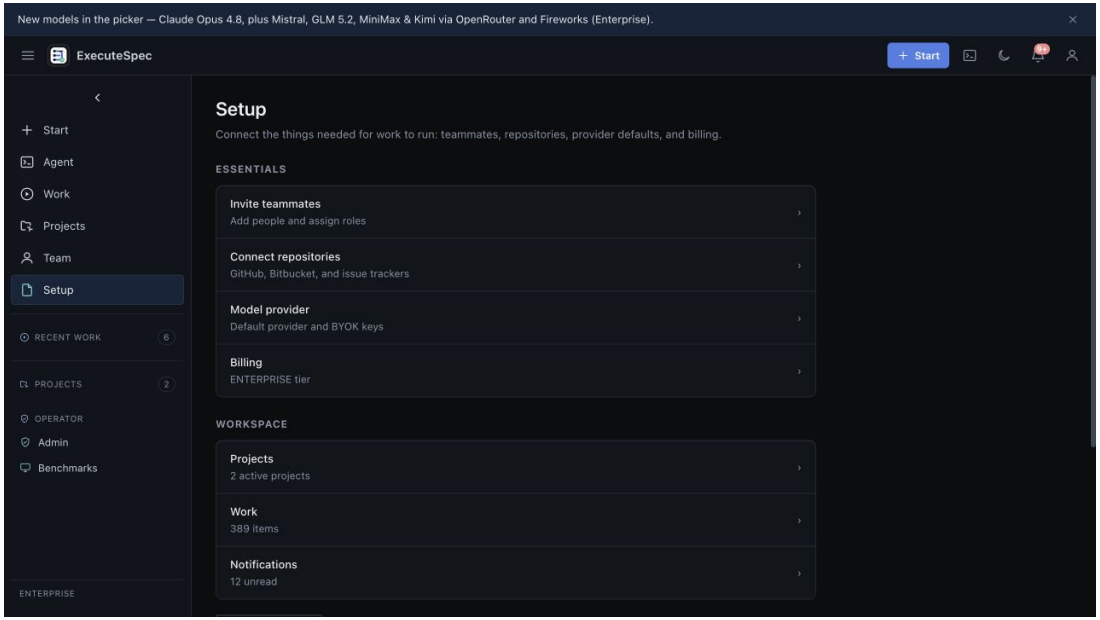
7. Run detail for local execution

- Good: strongest surface; status, local execution, provider, cost, request, and tabs are visible.
- Risk: post-run action hierarchy should lead with diff, verify, evidence, and revert.



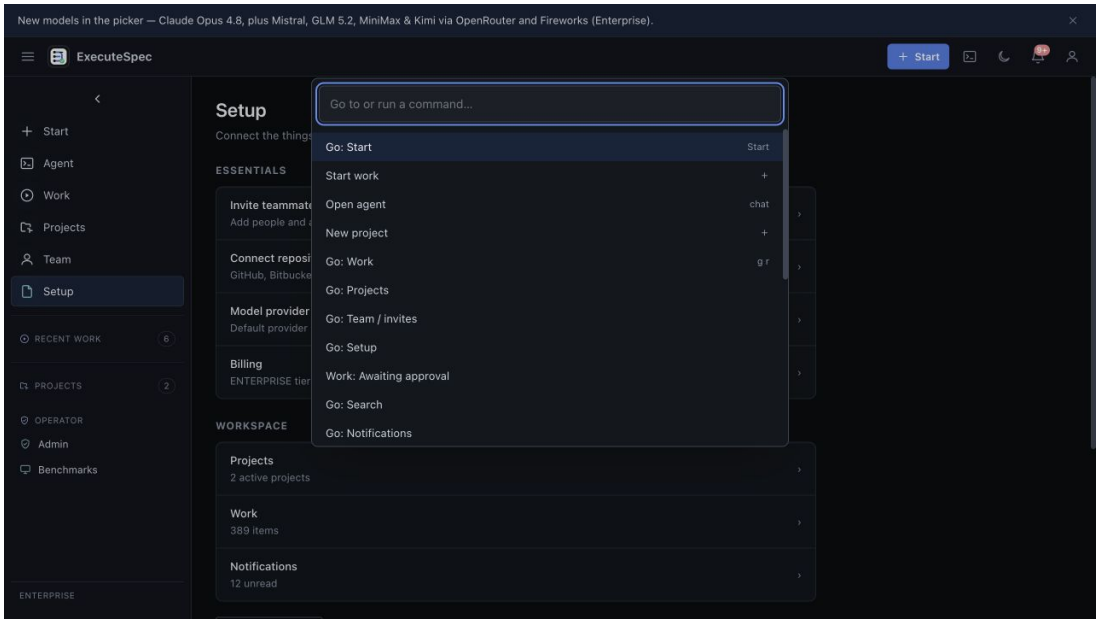
8. Projects list

- Good: clean and honest project inventory.
- Risk: sparse cards do not prove repo intelligence or readiness.



9. Setup surface

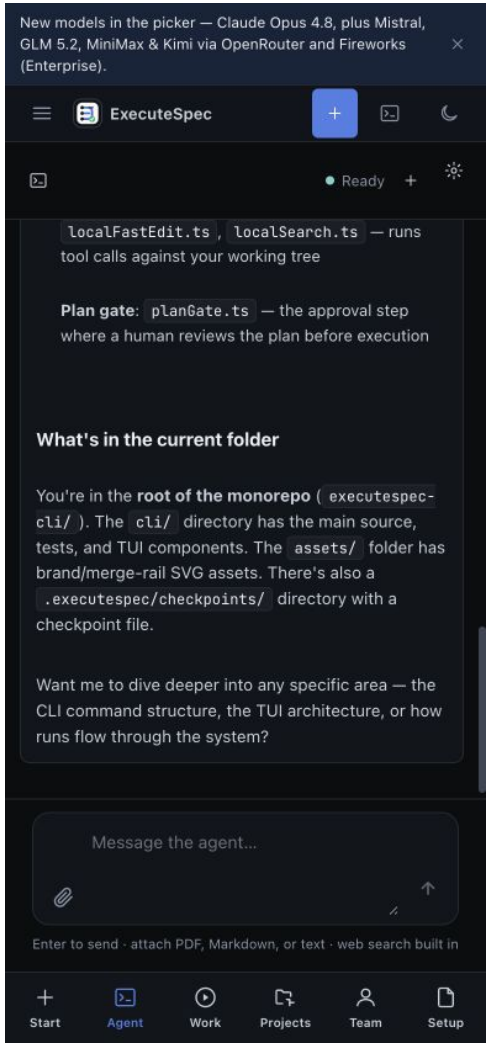
- Good: strong grouping for team, providers, billing, projects, work, notifications.
- Risk: setup still reads administrative, not operational readiness.

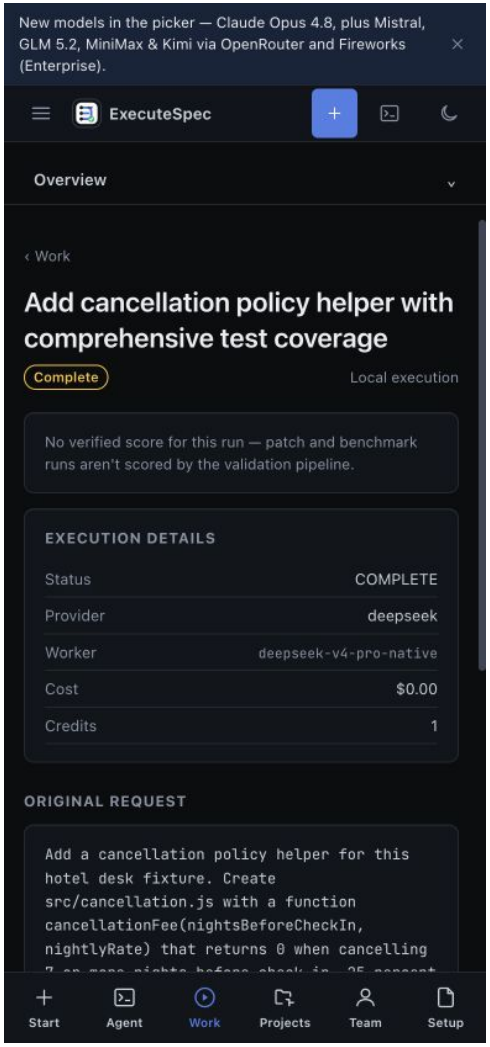


10. Command palette

- Good: useful operator shortcut and app-shell quality signal.
- Risk: missing local-hands commands such as inspect diff, verify, approve shell, revert.

6. Mobile Evidence





Mobile verdict

- The mobile shell is functional, which is useful for monitoring and approval checks.
- The announcement banner consumes too much of the task viewport.
- Run detail proof and actions land too far below the fold.
- Dense transcript and muted text increase cognitive load; this is not yet comfortable for serious mobile review.

7. Priority Fixes

P0	Add a local-hands launcher/cockpit path in web: prompt, repo, local/session state, permission queue, live tool, changed files, tests, cost.	This is the difference between a console and a coding-agent replacement.
P0	For local-execution run detail, make primary actions: View diff, Run verification, Open evidence, Revert safely, Continue in CLI/TUI.	The current actions are broad product actions; replacement users need proof actions first.
P0	Expose permission model before and during runs: read/edit/shell/build/test/git, once/always/reject, deny-wins, session scope.	Claude Code and serious local tools win trust here. Web must not hide it.
P1	Collapse or route-scope the announcement banner on active work and mobile.	It steals the highest-value pixels during actual work.
P1	Improve first-run agent state: no old transcript default, show capabilities, recent local run, next action, and safety boundary.	The product should not make a returning session look like the main product.
P1	Add command palette entries for local-hands actions: approve pending command, inspect changed files, run tests, view evidence, revert.	The shell already has the right mechanism; add the commands users need.
P2	Make projects cards show readiness: repos connected, docs indexed, last analysis, stale state, open PRs, local eligibility.	Projects should signal whether the agent has enough context to work.

Product stance: keep the dense operator UI. Do not turn this into a marketing dashboard. The right move is to make the coding loop visible, not decorative.

8. Accessibility And QA Limits

- **Visible accessibility positives:** top-bar buttons in ConsoleShell include aria labels/titles, main content receives focus on route changes, and the command palette is keyboard-oriented.
- **Visible accessibility risks:** small muted text, dense mobile layouts, banner height, and below-fold proof/actions on mobile.
- **Not tested:** full screen-reader traversal, focus order across every route, measured color contrast, 200 percent zoom/reflow, reduced motion, and end-to-end keyboard-only completion.
- **QA limit:** this report used an authenticated local frontend against deployed dev API. It did not deploy a new backend or run GitHub CI.

9. Sources

OpenAI Codex Web	https://developers.openai.com/codex/cloud
Claude Code overview	https://code.claude.com/docs/en/overview
Claude Code GitHub Actions	https://code.claude.com/docs/en/github-actions
Cursor Agent overview	https://cursor.com/docs/agent/overview
Cursor Bugbot	https://cursor.com/docs/bugbot
Devin overview	https://docs.devin.ai/get-started/devin-intro
Replit Agent	https://docs.replit.com/replitai/agent